VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Hospital Patient Management System

A9602 - Object Oriented Programming through Java Laboratory

Submitted by

Batch No. 3

CH. BEULAH 25881A05CK

Course Facilitator

Mr. D. Ganesh
Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

Contents

1 Introduction . . . 3
2 Literature Review . . . 3
3 Objectives . . . 4
4 Methodology . . . 4
4.1 Architectural Diagram . . . 4
4.2 Hardware and Software Requirements . . . 5
4.3 Working Principle . . . 5
4.4 Software Implementation . . . 6
5 Results and Discussion . . . 13
5.1 Limitations and Future Enhancements . . . 19
6 Mapping POs and SDGs . . . 19
6.1 Program Outcomes (POs) Mapping . . . 19
6.2 Sustainable Development Goals (SDGs) Mapping . . . 19
7 Conclusion . . . 19
Bibliography . . . 22

Abstract

Hospitals and clinics handle a large number of patient records every day. When these records are maintained in paper registers, searching for a patient's details takes time, the same information is written again and again, and billing mistakes are common at the time of discharge. This project presents a Hospital Patient Management System developed in Java as a menu-driven console application. The system allows the hospital staff to register new patients, display all registered patients, search for a patient by ID or by name, update or delete a patient record, view the list of doctors, assign a doctor to a patient, book and view appointments, admit a patient into a General, Private or ICU room, discharge a patient with an automatically calculated bill, and view a summary of patients, appointments and revenue. All records are saved to text files after every change, so the data is preserved when the program is closed and reopened.

The project is built using basic object oriented programming concepts. A `Person` class holds the details common to all persons in the hospital, and the `Patient` and `Doctor` classes extend it through inheritance. Patient, doctor and appointment records are stored in `ArrayList` collections, file handling is used to save them in text files, and exception handling is used to protect the program from invalid input such as text entered in place of a number. The bill at discharge is calculated from a fixed consultation fee and the room charge per day multiplied by the number of days stayed. The program was compiled and executed, and 30 test cases covering all its functions, invalid inputs and data persistence across restarts were performed; the results matched the expected output in every test case.

Keywords: Hospital management, patient records, Java, object oriented programming, console application, billing system, file handling, ArrayList, healthcare information system.

1 Introduction

A hospital is an organisation where information has to be handled quickly and correctly. Every day, patients are registered at the reception, examined by doctors, admitted into wards and rooms, and finally discharged after their treatment. Details such as the patient's name, age, disease, treating doctor and room number are needed again and again during this process. In many small hospitals and clinics, all of this information is still maintained manually in paper registers.

Manual record keeping has several problems. Searching a register for one patient's record is slow, especially when the number of patients is large. Handwritten records can be misread or lost, and the same patient details are rewritten at registration, admission and discharge. Billing is another area where manual work causes errors, because the total amount has to be calculated by hand from the room charges and the number of days stayed.

Computerised hospital information systems solve these problems by storing records in a structured form and providing fast search and automatic calculation. This project implements a small-scale version of such a system using Java. The Hospital Patient Management System is a console-based, menu-driven program that covers the basic workflow of a hospital: patient registration, doctor assignment, appointment booking, admission, and discharge with billing. Records are stored permanently in text files, so the system can be closed and reopened without losing data. It demonstrates how the object oriented concepts taught in the course — classes, objects, inheritance, encapsulation and exception handling — can be applied to a real-world problem of moderate size.

Java is a suitable language for this project because it is platform independent, strongly typed, and fully object oriented. The program uses only the Java standard library (`java.util` package), so it runs on any computer with a Java Development Kit (JDK) installed, without any additional software or database server.

2 Literature Review

The management of patient information has been studied for a long time as part of the wider subject of health care information systems. Wager, Lee and Glaser, in their book on health care information systems, describe how hospitals moved from paper-based records to electronic systems in order to improve the speed of information access, reduce duplication of work and support better clinical and administrative decisions [4]. Even a simple electronic record system provides clear advantages over registers: a record stored once can be retrieved instantly from anywhere in the organisation, and calculations such as billing can be performed automatically and consistently.

Standard references on health informatics point out that a hospital information system is normally built around the patient record, and that functions such as registration, admission, discharge and billing are the core administrative modules of such a system [4]. The present project follows the same idea at a laboratory scale: the patient is the central object of the system, and registration, admission, discharge and billing are implemented as operations on patient records.

On the implementation side, object oriented programming is the standard approach taught for building such record-management applications in Java. Schildt explains how inheritance allows a general class to be specialised into related classes that share common data and behaviour [1], and Deitel and Deitel demonstrate the use of collections such as `ArrayList` for storing and processing groups of objects in place of fixed-size arrays [3]. Balagurusamy similarly presents menu-driven console programs as a practical way of combining classes, methods and exception handling in student projects [2]. The design of this project — a `Person` base class specialised into `Patient` and `Doctor`, records stored in `ArrayLists`, and a menu-driven main class — is

based on these established practices.

3 Objectives

- To develop a menu-driven Java program that computerises the basic patient record work of a hospital, replacing manual paper registers.
- To register patients with their personal and medical details and store the records in an `ArrayList` so that they can be displayed and searched quickly by patient ID and by patient name.
- To update and delete patient records safely, refusing deletion of a patient who is currently admitted.
- To maintain a list of doctors with their specializations and to assign a suitable doctor to each patient.
- To book appointments for patients with a selected doctor, date and time, and to display all booked appointments.
- To admit patients into General, Private or ICU rooms and to record their admission status and room number.
- To generate the discharge bill automatically from the consultation fee and the room charges for the number of days stayed, reducing manual calculation errors.
- To save all records, ID counters and the collected revenue in text files using file handling, so that data is preserved between runs of the program.
- To display a hospital summary showing the number of registered patients, admitted patients, appointments and total revenue collected.
- To apply object oriented concepts — classes, objects, inheritance, encapsulation and exception handling — in a complete, working application, and to handle invalid input without the program terminating abnormally.

4 Methodology

4.1 Architectural Diagram

The system follows a simple layered structure, as shown in Figure 1. The hospital staff member (user) interacts with the menu-driven console interface provided by the main class, `HospitalManagementSystem`. Each menu choice calls a method that performs one task: patient management (registration, display, search, update and deletion), doctor management (doctor list and assignment), appointments (booking and viewing), or admission and billing (admission, discharge, bill calculation and the hospital summary). Patient and appointment records are held in `ArrayList` collections while the program runs and are saved to text files after every change, so they are available again when the program is restarted.

The class structure supports this architecture. The `Person` class is the base class; `Patient` and `Doctor` inherit from it, and a separate `Appointment` class represents one booking. The main class creates and manages objects of these classes and provides every operation offered in the menu.

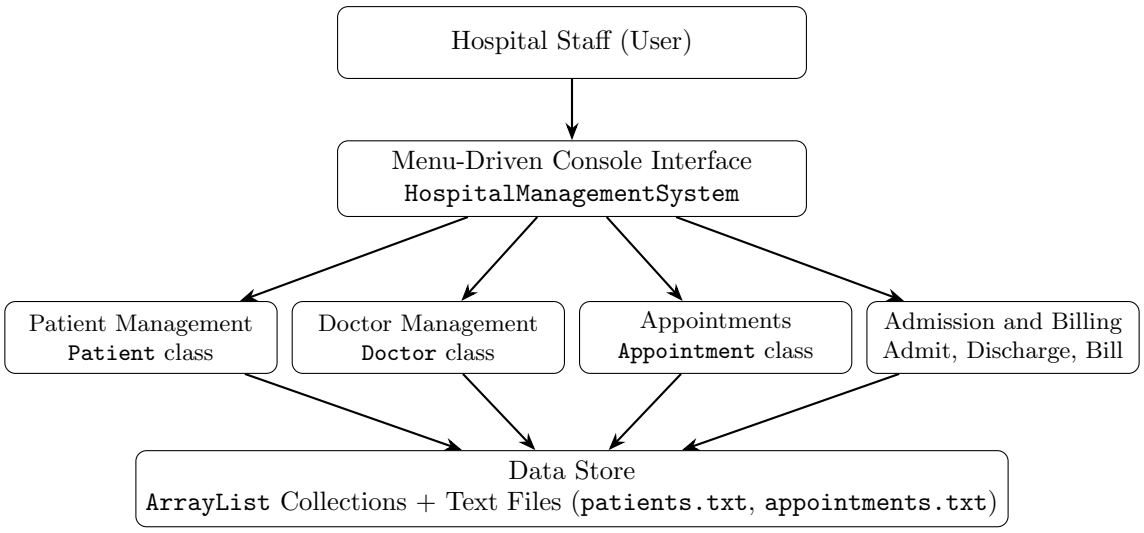


Figure 1: Architectural Diagram of the Hospital Patient Management System

4.2 Hardware and Software Requirements

Hardware Requirements:

- Computer or laptop with a minimum of 4 GB RAM
- At least 100 MB of free disk space for the project and report files

Software Requirements:

- Operating system: Windows, Linux or macOS
- Java Development Kit (JDK) 8 or newer (developed and tested on OpenJDK 17)
- Code editor: VS Code, IntelliJ IDEA, Eclipse or any Java environment
- Command prompt or terminal to compile and run the program
- No external libraries are required; the project uses only the Java standard library
- Data files `patients.txt` and `appointments.txt` are created automatically in the working folder when the program runs

4.3 Working Principle

The working of the Hospital Patient Management System can be understood as a sequence of simple steps:

1. The program starts by loading a small list of doctors (with their specializations) into an `ArrayList`. It then loads any patient and appointment records saved during earlier runs from the text files `patients.txt` and `appointments.txt`, along with the next patient ID, next room number, next appointment ID and the total revenue collected so far. On the very first run these files do not exist, so the program starts with no records. It then displays the main menu with fourteen options.

2. When the user chooses Register New Patient, the program reads the patient's name, age, gender, phone number and disease. The age is checked: if it is not a number, or not between 1 and 120, the patient is not registered and a message is shown.

3. Each registered patient is given a unique patient ID automatically, starting from 101, and the `Patient` object is added to the patients `ArrayList`. Because the next ID is saved in the data file, IDs are never reused after a restart.

4. The Display All Patients option prints the complete details of every registered patient. The Search options find patients by ID or by name (the name search does not depend on capital or small letters), or report that no matching patient was found.

5. The Update option changes a patient's phone number and disease, and the Delete option removes a patient's record. Deletion is refused for a patient who is currently admitted, and the deleted patient's appointments are removed along with the record.

6. The Display All Doctors option prints the doctor list. Using the Assign Doctor option, the user enters a patient ID and a doctor ID, and the doctor's name is stored in the patient record.

7. Using the Book Appointment option, the user selects a patient and a doctor and enters a date and time; the program creates an `Appointment` object with the next appointment ID. The View Appointments option prints all bookings with the patient and doctor names.

8. When a patient is admitted, the user selects the room type (General, Private or ICU). The program assigns the next free room number automatically, starting from 201, and marks the patient as admitted. If the room type choice is anything other than 1, 2 or 3, the admission is cancelled with a message, and a patient who is already admitted cannot be admitted again.

9. At discharge, the user enters the number of days stayed. The program calculates the bill as a fixed consultation fee of Rs. 500 plus the room charge per day (General Rs. 1,000, Private Rs. 2,500, ICU Rs. 5,000) multiplied by the number of days, prints the bill, adds the amount to the total revenue, and marks the patient as discharged. The number of days must be at least 1; an invalid value cancels the discharge and the patient remains admitted. Discharge of a patient who is not admitted is also refused.

10. After every operation that changes data (register, update, delete, assign, admit, discharge or book), the complete records are saved back to the text files, so no completed work is lost even if the program is closed suddenly.

11. The Hospital Summary option prints the total number of registered patients, the number of patients admitted at present, the number of appointments booked and the total revenue collected from discharge bills.

12. Steps 2 to 11 can be repeated until the user chooses the Exit option.

4.4 Software Implementation

The implementation is organised into five Java files, each with a single responsibility, as listed in Table 1.

Algorithm: Hospital Patient Management System

1. Start the program, add the sample doctors to the doctors list, and load the saved patients, appointments and counters from the text files (on the first run, start empty).

2. Display the menu and read the user's choice. If the choice is not a number, display an error message and show the menu again.

3. If the choice is Register: read and validate the patient details, create a `Patient` object with the next patient ID, and add it to the list.

Table 1: Classes Used in the System

Class	Responsibility
`Person`	Base class holding the details common to every person: name, age, gender and phone number, with getter methods and a method to display them.
`Patient`	Extends `Person`. Adds patient ID, disease, assigned doctor, admission status and room details. Provides methods to admit, discharge and calculate the bill.
`Doctor`	Extends `Person`. Adds doctor ID and specialization, and a method to display the doctor in one line.
`Appointment`	Represents one appointment: appointment ID, patient ID, doctor name, date and time, with a method to display the booking.
`HospitalManagementSystem`	Main class. Displays the menu, reads the user's choice, and performs registration, display, search, update, deletion, doctor assignment, appointment booking, admission, discharge and the summary using `ArrayLists`. It also saves and loads all records from text files.

4. If the choice is Display or Search: print all patient records, or find and print the records matching the entered ID or name.

5. If the choice is Update or Delete: change the patient's phone number and disease, or remove the record (deletion is refused for an admitted patient).

6. If the choice is Assign Doctor: find the patient and the doctor by their IDs and store the doctor's name in the patient record.

7. If the choice is Book Appointment: select the patient and doctor, read the date and time, and store an `Appointment` object with the next appointment ID.

8. If the choice is Admit: select the room type, assign the next room number, and mark the patient as admitted.

9. If the choice is Discharge: read the number of days stayed, calculate and print the bill, add it to the total revenue, and mark the patient as discharged.

10. If the choice is Summary: count the registered patients, admitted patients and appointments, and print them with the total revenue.

11. After every change, save all records and counters to the text files. Repeat from step 2 until the user chooses Exit.

The important parts of the code are given below. The `Person` class (Listing 1) shows encapsulation: the data members are `private` and are accessed through getter methods. The `Patient` class (Listing 2) shows inheritance using the `extends` keyword and the `super()` call to the base class constructor, together with the bill calculation method. The `Doctor` class is given in Listing 3, and the `Appointment` class in Listing 4. All listings are taken directly from the project source files.

```
1 // Person.java - Base class holding details common to patients and doctors
2 public class Person {
```

```
    private String name;
    private int age;
    private String gender;
    private String phone;

    public Person(String name, int age, String gender, String phone) {
        this.name = name;
        this.age = age;
        this.gender = gender;
        this.phone = phone;
    }

    public String getName()   { return name; }
    public int getAge()       { return age; }
    public String getGender() { return gender; }
    public String getPhone()  { return phone; }

    public void setPhone(String phone) { this.phone = phone; }

    public void displayBasicDetails() {
        System.out.println("Name   : " + name);
        System.out.println("Age    : " + age);
        System.out.println("Gender : " + gender);
        System.out.println("Phone  : " + phone);
    }
}
```

Listing 1: Person.java – Base Class

```
// Patient.java - Represents one patient registered in the hospital
public class Patient extends Person {
    private int patientId;
    private String disease;
    private String doctorName;
    private boolean admitted;
    private String roomType;
    private int roomNo;

    public Patient(int patientId, String name, int age, String gender,
                   String phone, String disease) {
        super(name, age, gender, phone);   // call the Person constructor
        this.patientId = patientId;
        this.disease = disease;
        this.doctorName = "Not Assigned";
        this.admitted = false;
        this.roomType = "-";
        this.roomNo = 0;
    }

    public int getPatientId()      { return patientId; }
    public String getDisease()     { return disease; }
    public String getDoctorName()  { return doctorName; }
    public boolean isAdmitted()    { return admitted; }
    public String getRoomType()    { return roomType; }
    public int getRoomNo()         { return roomNo; }

    public void setDisease(String disease) {
        this.disease = disease;
    }

    public void setDoctorName(String doctorName) {
        this.doctorName = doctorName;
    }
```

```
35
36      public void admit(String roomType, int roomNo) {
37          this.roomType = roomType;
38          this.roomNo = roomNo;
39          this.admitted = true;
40      }
41
42      public void discharge() {
43          this.admitted = false;
44      }
45
46      // Room charge for one day depends on the type of room
47      public int roomChargePerDay() {
48          if (roomType.equals("Private")) {
49              return 2500;
50          } else if (roomType.equals("ICU")) {
51              return 5000;
52          } else {   // General room
53              return 1000;
54          }
55      }
56
57      // Total bill = consultation fee + (room charge per day x days stayed)
58      public int calculateBill(int days) {
59          int consultationFee = 500;
60          return consultationFee + (roomChargePerDay() * days);
61      }
62
63      public void displayPatient() {
64          System.out.println("Patient ID : " + patientId);
65          displayBasicDetails();
66          System.out.println("Disease    : " + disease);
67          System.out.println("Doctor     : " + doctorName);
68          if (admitted) {
69              System.out.println("Status     : Admitted (" + roomType
70                                 + ", Room No. " + roomNo + ")");
71          } else {
72              System.out.println("Status     : Not Admitted");
73          }
74          System.out.println("----------------------------------------");
75      }
76  }
```

Listing 2: Patient.java – Patient Class with Bill Calculation

```
1   // Doctor.java - Represents one doctor available in the hospital
2   public class Doctor extends Person {
3       private int doctorId;
4       private String specialization;
5
6       public Doctor(int doctorId, String name, int age, String gender,
7                     String phone, String specialization) {
8           super(name, age, gender, phone);
9           this.doctorId = doctorId;
10          this.specialization = specialization;
11      }
12
13      public int getDoctorId()          { return doctorId; }
14      public String getSpecialization() { return specialization; }
15
16      public void displayDoctor() {
17          System.out.printf("%-4d %-20s %-18s %s%n",
18                  doctorId, getName(), specialization, getPhone());
```

```
    }
}
```

Listing 3: Doctor.java – Doctor Class

```
// Appointment.java - Represents one appointment booked for a patient
public class Appointment {
    private int appointmentId;
    private int patientId;
    private String doctorName;
    private String date;
    private String time;

    public Appointment(int appointmentId, int patientId,
                       String doctorName, String date, String time) {
        this.appointmentId = appointmentId;
        this.patientId = patientId;
        this.doctorName = doctorName;
        this.date = date;
        this.time = time;
    }

    public int getAppointmentId()  { return appointmentId; }
    public int getPatientId()      { return patientId; }
    public String getDoctorName()  { return doctorName; }
    public String getDate()        { return date; }
    public String getTime()        { return time; }

    public void displayAppointment(String patientName) {
        System.out.println("Appointment ID : " + appointmentId);
        System.out.println("Patient        : " + patientName
                           + " (ID " + patientId + ")");
        System.out.println("Doctor         : " + doctorName);
        System.out.println("Date           : " + date);
        System.out.println("Time           : " + time);
        System.out.println("----------------------------------------");
    }
}
```

Listing 4: Appointment.java – Appointment Class

The main class displays the menu in a `do-while` loop and uses a `switch` statement to call the method for the chosen option. The menu loop (Listing 5) and the discharge method, which prints the bill (Listing 6), are shown below. Input errors are handled with `try-catch` blocks for `InputMismatchException`, so that entering text where a number is expected does not stop the program.

```
    public static void main(String[] args) {
        loadSampleDoctors();
        loadData();
        int choice = 0;
        do {
            displayMenu();
            try {
                choice = scanner.nextInt();
                scanner.nextLine();   // clear the pending newline
            } catch (InputMismatchException e) {
                System.out.println("Invalid input! Please enter a number.");
                scanner.nextLine();
                continue;
            }
            switch (choice) {
                case 1:  registerPatient();    break;
```

```
                case 2:  displayAllPatients(); break;
                case 3:  searchPatientById();  break;
                case 4:  searchPatientByName(); break;
                case 5:  updatePatient();      break;
                case 6:  deletePatient();      break;
                case 7:  displayAllDoctors();  break;
                case 8:  assignDoctor();       break;
                case 9:  bookAppointment();    break;
                case 10: viewAppointments();   break;
                case 11: admitPatient();       break;
                case 12: dischargePatient();   break;
                case 13: viewSummary();        break;
                case 14: System.out.println("Thank you. Exiting the system.");
                    break;
                default: System.out.println("Invalid choice! Please try again."
                    );
            }
        } while (choice != 14);
    }
```

Listing 5: HospitalManagementSystem.java – Menu Loop in the Main Method

```
    static void dischargePatient() {
        System.out.print("Enter patient ID to discharge: ");
        int id = readInt();
        if (id == -1) {
            System.out.println("Invalid input! Patient ID must be a number.");
            return;
        }
        Patient p = findPatient(id);
        if (p == null) {
            System.out.println("Patient with ID " + id + " not found.");
            return;
        }
        if (!p.isAdmitted()) {
            System.out.println("Patient is not admitted, so discharge is not
                possible.");
            return;
        }
        System.out.print("Enter number of days stayed: ");
        int days = readInt();
        if (days == -1) {
            System.out.println("Invalid input! Number of days must be a number.
                 Discharge cancelled.");
            return;
        }
        if (days <= 0) {
            System.out.println("Invalid number of days! Days must be at least
                1. Discharge cancelled.");
            return;
        }
        int roomCharges = p.roomChargePerDay() * days;
        int total = p.calculateBill(days);
        System.out.println("\n---------- DISCHARGE BILL ----------");
        System.out.println("Patient ID   : " + p.getPatientId());
        System.out.println("Name         : " + p.getName());
        System.out.println("Room Type    : " + p.getRoomType() + " (Room No. "
            + p.getRoomNo() + ")");
        System.out.println("Days Stayed  : " + days);
        System.out.println("Consultation Fee         : Rs. 500");
        System.out.println("Room Charges (" + days + " x " + p.roomChargePerDay
            () + ") : Rs. " + roomCharges);
        System.out.println("Total Bill               : Rs. " + total);
```

```
        System.out.println("---------------------------------------");
        p.discharge();
        totalRevenue = totalRevenue + total;
        saveAll();
        System.out.println("Patient discharged successfully.");
    }
```

Listing 6: HospitalManagementSystem.java – Discharge and Bill Generation Method

Numerical input in the search, assignment, admission and discharge methods is read through a small helper method, `readInt()` (Listing 7), which catches `InputMismatchException` and returns -1 for non-numeric input; the calling method then rejects the operation with a message instead of the program stopping suddenly.

```
    // Reads one integer safely. Returns -1 when the input is not a
    // number, so the calling method can reject it and return to the menu.
    static int readInt() {
        try {
            int value = scanner.nextInt();
            scanner.nextLine();   // clear the pending newline
            return value;
        } catch (InputMismatchException e) {
            scanner.nextLine();   // discard the wrong input
            return -1;
        }
    }
```

Listing 7: HospitalManagementSystem.java – Safe Integer Input Helper

File handling is implemented with `BufferedReader` and `BufferedWriter`. Each record is written on one line with its fields separated by the `|` character, and the first line of `patients.txt` stores the counters and total revenue, as shown in the `savePatients()` method in Listing 8. The `loadData()` method reads the same format back at start-up and rebuilds the objects, calling `admit()` for patients who were admitted when the program was closed.

```
    // -------------------- File handling --------------------
    // Records are saved in simple text files, one record per line,
    // with the fields separated by the '|' character. The first line
    // of patients.txt stores the counters and the total revenue.

    static void savePatients() {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(PATIENT_FILE
            ))) {
            bw.write("#META|" + nextPatientId + "|" + nextRoomNo + "|"
                     + nextAppointmentId + "|" + totalRevenue);
            bw.newLine();
            for (Patient p : patients) {
                bw.write(p.getPatientId() + "|" + p.getName() + "|" + p.getAge
                    ()
                         + "|" + p.getGender() + "|" + p.getPhone() + "|" + p.
                             getDisease()
                         + "|" + p.getDoctorName() + "|" + p.isAdmitted()
                         + "|" + p.getRoomType() + "|" + p.getRoomNo());
                bw.newLine();
            }
        } catch (IOException e) {
            System.out.println("Warning: could not save patient data.");
        }
    }
```

Listing 8: HospitalManagementSystem.java – Saving Patient Records to a Text File

Patients are searched by a simple linear search method, `findPatient()`, which compares the entered ID with each patient's ID in the list and returns the matching object, or `null` when no patient has that ID. Registration validates the age before creating the object, as described in the working principle. The complete source code of all five classes is provided with this report in the source package.

5 Results and Discussion

The program was compiled using the Java compiler (`javac`, OpenJDK 17) without any errors, and it was tested in six runs using scripted input files, so that every test can be repeated exactly. Each run started in a fresh folder containing only the compiled classes, so it began with no saved data — except Run 5, whose two parts shared one folder to test persistence. In the first run, two patients were registered, the patient and doctor lists were displayed, a search was performed for an existing and a non-existing ID, doctors were assigned, one patient was admitted to a Private room and then discharged after three days, and an invalid menu choice was tested. The second and third runs tested invalid age input. The fourth run tested the admission and discharge edge cases: an invalid room type choice, admitting the same patient twice, discharging with zero days, discharge from a General room after two days, discharging the same patient twice, assigning a doctor to a non-existing patient, a non-numeric patient ID, and discharging a non-existing patient. In the fifth run, a patient was registered and admitted, the program was closed and started again in the same folder, and the records and the patient ID counter were found to be preserved. The sixth run tested search by name, updating a patient's details, booking and viewing appointments, deleting an admitted patient (refused), deleting a discharged patient, and the hospital summary. In total 30 test cases were executed, and all of them passed. The test cases and their results are summarised in Table 2.

Table 2: Test Cases and Results

TC	Test Case	Input	Expected / Actual Output	Result
TC01	Register a new patient	Choice 1; Ravi Kumar, 35, Fever	Patient registered, ID 101 assigned	Pass
TC02	Display all patients	Choice 2	Details of patients 101 and 102 displayed	Pass
TC03	Search existing patient by ID	Choice 3; ID 101	Record of patient 101 displayed	Pass
TC04	Search non-existing patient	Choice 3; ID 999	"Patient with ID 999 not found."	Pass
TC05	Display all doctors	Choice 7	Table of 4 doctors with specializations	Pass
TC06	Assign doctor	Choice 8; Patient 102, Doctor 4	Dr. Arjun Mehta assigned to Meena Sharma	Pass
TC07	Admit (Private) and discharge, 3 days	Choices 11 and 12; 3 days	Bill = 500 + (3 x 2500) = Rs. 8000	Pass
TC08	Invalid menu choice	Choice 99	"Invalid choice! Please try again."	Pass
TC09	Age entered as text	Age: "thirty"	"Invalid input! Age must be a number." Not registered	Pass
TC10	Age out of range	Age: 150	"Invalid age! Patient not registered."	Pass
TC11	Invalid room type at admission	Choice 11; room choice 9	"Invalid room type choice! Admission cancelled." Patient remains Not Admitted	Pass

TC	Test Case	Input	Expected / Actual Output	Result
TC12	Admit same patient twice	Choice 11 for admitted patient	"Patient is already admitted."	Pass
TC13	Discharge with 0 days	Choice 12; days 0	"Invalid number of days!" Discharge cancelled, patient stays admitted	Pass
TC14	Discharge, General room, 2 days	Choice 12; days 2	Bill = 500 + (2 x 1000) = Rs. 2500	Pass
TC15	Discharge a patient not admitted	Choice 12 repeated after discharge	"Patient is not admitted, so discharge is not possible."	Pass
TC16	Assign doctor to non-existing patient	Choice 8; Patient 999	"Patient with ID 999 not found."	Pass
TC17	Non-numeric patient ID in search	Choice 3; ID "abc"	"Invalid input! Patient ID must be a number." Program continues	Pass
TC18	Discharge non-existing patient	Choice 12; ID 999	"Patient with ID 999 not found."	Pass
TC19	Records survive a restart	Run 5: register, admit, exit, restart, display	Patient 101 still listed, still Admitted (General, Room 201)	Pass
TC20	ID counters survive a restart	Run 5: register a patient after restart	New patient registered with ID 102	Pass
TC21	Search by name (different case)	Choice 4; "anil gupta"	Record of Anil Gupta displayed	Pass
TC22	Search by a name that does not exist	Choice 4; "Nobody"	"No patient found with name Nobody."	Pass
TC23	Update patient details	Choice 5; new phone and disease	"Patient details updated successfully." Display shows the new values	Pass
TC24	Book an appointment	Choice 9; Patient 101, Doctor 4, 28-10-2026, 10:30 AM	"Appointment booked successfully. Appointment ID = 1"	Pass
TC25	Book appointment for non-existing patient	Choice 9; Patient 999	"Patient with ID 999 not found."	Pass
TC26	View all appointments	Choice 10	Appointment 1 listed with patient name, doctor, date and time	Pass
TC27	Delete an admitted patient	Choice 6; admitted patient	"Cannot delete an admitted patient. Discharge the patient first."	Pass
TC28	Discharge, General room, 1 day	Choice 12; days 1	Bill = 500 + (1 x 1000) = Rs. 1500	Pass
TC29	Delete a discharged patient	Choice 6; discharged patient	"Patient record deleted successfully."	Pass
TC30	Hospital summary	Choice 13 after Runs 6 operations	1 patient, 0 admitted, 1 appointment, revenue Rs. 1500	Pass

The display of the registered patients produced the following output:

```
---------- LIST OF PATIENTS ----------
Patient ID : 101
Name   : Ravi Kumar
Age    : 35
Gender : Male
Phone  : 9876543210
Disease    : Fever
```

```
Doctor     : Not Assigned
Status     : Not Admitted
----------------------------------------
Patient ID : 102
Name   : Meena Sharma
Age    : 28
Gender : Female
Phone  : 9876501234
Disease    : Fracture
Doctor     : Not Assigned
Status     : Not Admitted
----------------------------------------
```

The doctor list, from which a doctor is selected for assignment, is displayed as shown below. In the test run, assigning doctor ID 4 to patient 102 printed the message "Doctor Dr. Arjun Mehta assigned to patient Meena Sharma."

```
---------- LIST OF DOCTORS ----------
ID   Name                Specialization     Phone
1    Dr. Ananya Rao      Cardiology         9000000001
2    Dr. Vikram Reddy    General Medicine   9000000002
3    Dr. Sneha Patel     Pediatrics         9000000003
4    Dr. Arjun Mehta     Orthopedics        9000000004
```

The bill generated at the discharge of patient 102 (Private room, three days) is shown below. The total, Rs. 8000, agrees with the hand calculation: Rs. 500 consultation fee + 3 days x Rs. 2500 room charge = Rs. 8000.

```
---------- DISCHARGE BILL ----------
Patient ID   : 102
Name         : Meena Sharma
Room Type    : Private (Room No. 201)
Days Stayed  : 3
Consultation Fee         : Rs. 500
Room Charges (3 x 2500) : Rs. 7500
Total Bill               : Rs. 8000
----------------------------------------
Patient discharged successfully.
```

Some actual lines from the output of the fourth (edge-case) test run are given below. Each invalid operation was rejected with a message, and the program continued to the menu in every case:

```
Enter room type choice: Invalid room type choice! Admission cancelled.
Enter your choice: Enter patient ID to admit: Patient is already admitted.
Enter your choice: Enter patient ID to search: Invalid input! Patient ID must be a number.
Enter your choice: Enter patient ID to discharge: Patient with ID 999 not found.
```

In the same run, discharge from a General room after two days produced a bill of Rs. 2500, agreeing with the hand calculation Rs. 500 + (2 x Rs. 1000).

Screenshots taken during the actual execution of the program in a terminal are shown in Figures 2, 3 and 4. Each screenshot is from a separate run of the program, in which the patient was registered at the start of that run.

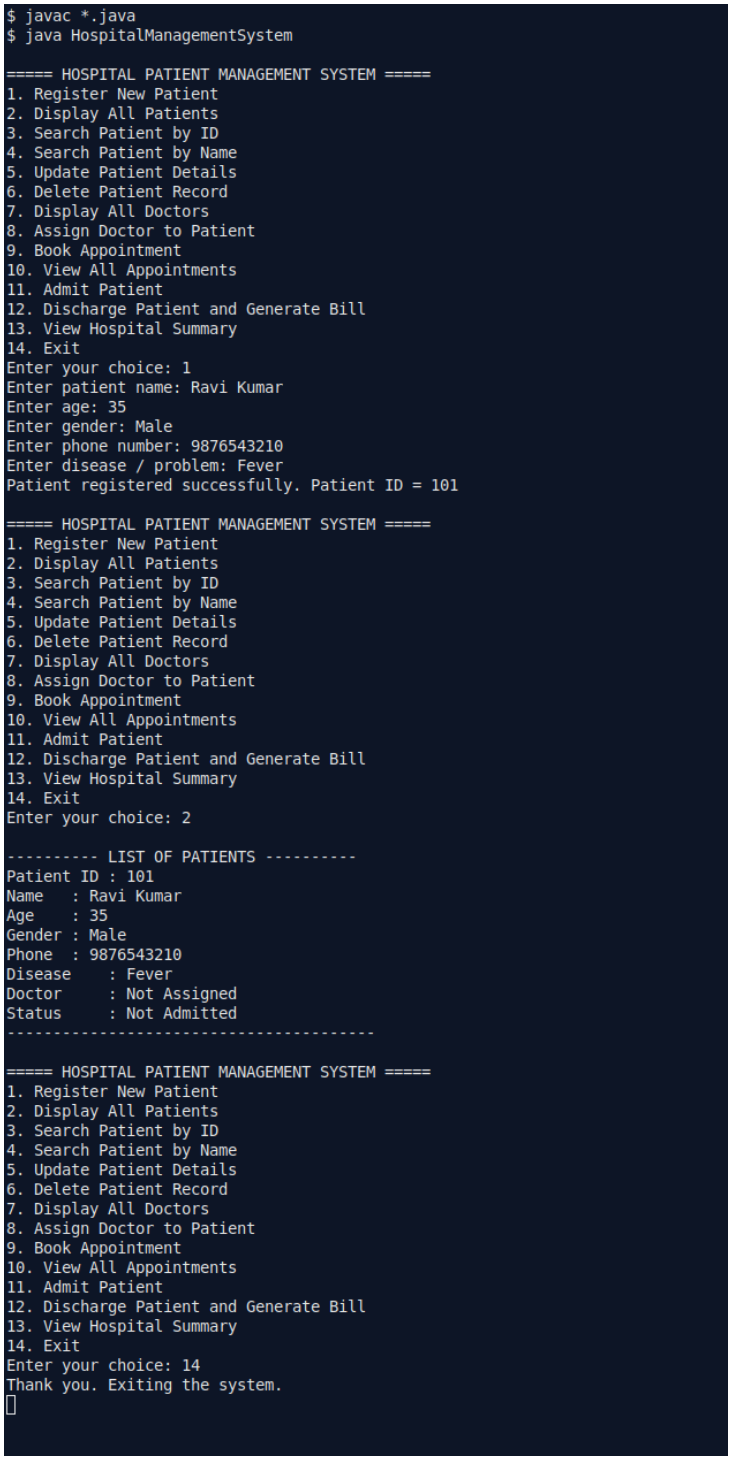


Figure 2: Screenshot of Program Execution – Patient Registration and Patient List

Figure 2 shows the main menu, the registration of a patient (Ravi Kumar) with the automatically assigned patient ID 101, and the patient list displayed using menu option 2.

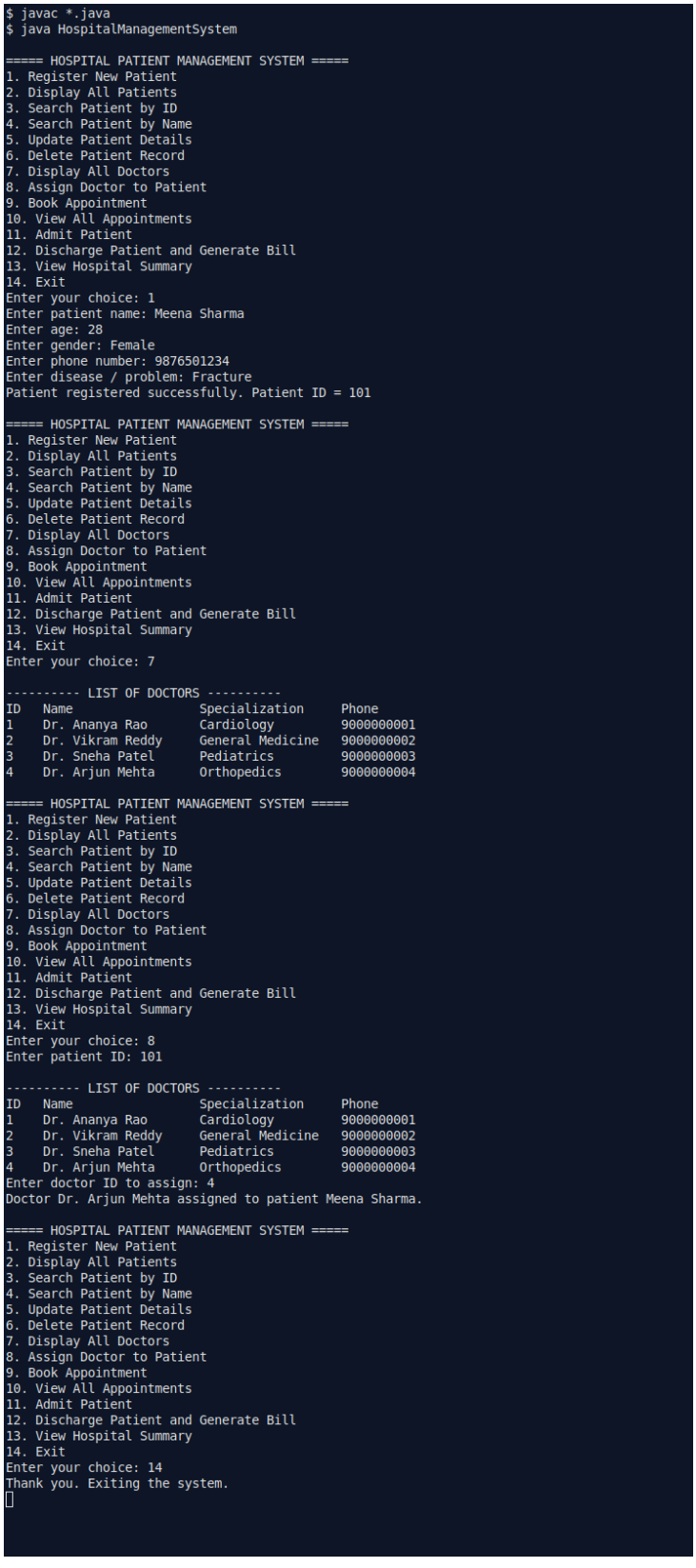


Figure 3: Screenshot of Program Execution – Doctor List and Doctor Assignment

Figure 3 shows the list of doctors with their specializations (menu option 7) and the assignment of Dr. Arjun Mehta to the patient using menu option 8.

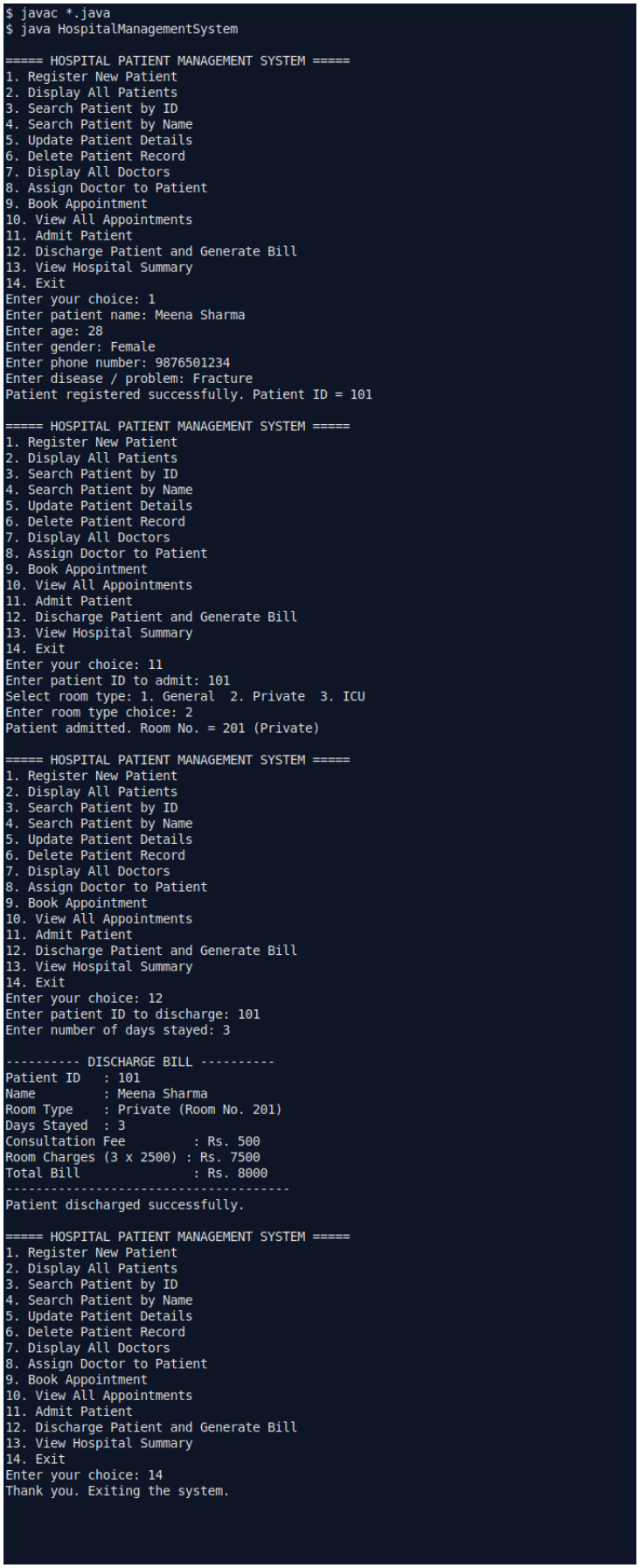


Figure 4: Screenshot of Program Execution – Admission and Discharge Bill

Figure 4 shows the admission of the patient to a Private room (Room No. 201) and the discharge bill generated after three days. The total bill of Rs. 8000 agrees with the hand calculation Rs. 500 + (3 x Rs. 2500).

In the persistence test (Run 5), the patient registered and admitted in the first part was displayed correctly when the program was started again, with the status "Admitted (General, Room No. 201)", and the next patient registered after the restart received ID 102, showing that both the records and the ID counters are stored in the data files. The hospital summary printed at the end of Run 6, after one discharge of Rs. 1500, one remaining patient and one appointment, is shown below:

```
---------- HOSPITAL SUMMARY ----------
Total Registered Patients : 1
Patients Admitted Now     : 0
Total Appointments Booked : 1
Total Revenue Collected   : Rs. 1500
--------------------------------------
```

The results show that all the main functions of the system — registration, display, search, doctor assignment, admission and discharge with billing — work as expected, that invalid inputs and invalid operations (wrong room type, zero days, repeated admission or discharge, non-existing IDs) are rejected without the program stopping suddenly, and that no invalid operation is carried out silently. Searching by patient ID is much faster than searching a paper register, and the bill is calculated by the program itself, so the chances of a calculation mistake at discharge are removed.

5.1 Limitations and Future Enhancements

The present system stores records in simple text files. This is easy to understand, but the files give no protection to the data, and a field value cannot contain the '—' separator character used in the file format. Appointment dates and times are accepted as typed text, without calendar validation or clash checking between two appointments of the same doctor. It is also a single-user console application without a login facility. These limitations can be removed in future work: records can be stored in a database through JDBC, a graphical user interface can be developed using Swing, and more modules such as pharmacy and laboratory records, a separate doctor login and appointment reminders can be added.

6 Mapping POs and SDGs

6.1 Program Outcomes (POs) Mapping

6.2 Sustainable Development Goals (SDGs) Mapping

7 Conclusion

This project successfully implements a Hospital Patient Management System in Java, replacing the slow and error-prone manual registers used for patient records. The system registers patients, maintains doctor details, assigns doctors, books appointments, admits patients into rooms, generates the discharge bill automatically, shows a summary of patients, appointments and revenue, and saves all records in text files so that no data is lost between runs. Object oriented concepts — inheritance through the `Person`, `Patient` and `Doctor` classes, encapsulation of data, and exception handling for invalid input — form the foundation of the program, and testing confirmed that every function works as expected. The system is simple, low-cost and easy to use. Future improvements can add permanent storage through files or a database,

Table 3: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	**Program Outcome**	**Relevance**
PO1	Engineering Knowledge	Applied object oriented programming concepts and Java language fundamentals to build the system.
PO2	Problem Analysis	Identified the problems of manual, paper-based patient record keeping in hospitals.
PO3	Design/Development of Solutions	Designed and developed a working menu-driven system for registration, admission and billing.
PO5	Modern Tool Usage	Used the Java Development Kit (JDK) and a code editor to develop, compile and test the program.
PO6	Engineer and Society	Provides a practical solution that helps hospitals serve patients faster and with fewer errors.

a graphical user interface, and additional hospital modules to make it suitable for real hospital use.

Table 4: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	**Goal**	**Relevance**
SDG 3	Good Health and Well-Being	Faster and more accurate patient records support better and safer health care services.
SDG 9	Industry, Innovation, and Infrastructure	Promotes digital infrastructure in hospitals and clinics, including small facilities.
SDG 10	Reduced Inequalities	A low-cost system that runs on ordinary computers can be adopted by small and rural hospitals as well.

Bibliography

[1] H. Schildt, *Java: The Complete Reference*, 12th ed. McGraw Hill, 2021.

[2] E. Balagurusamy, *Programming with Java*, 6th ed. McGraw Hill, 2019.

[3] P. Deitel and H. Deitel, *Java How to Program, Early Objects*, 11th ed. Pearson, 2018.

[4] K. A. Wager, F. W. Lee, and J. P. Glaser, *Health Care Information Systems: A Practical Approach for Health Care Management*, 4th ed. Jossey-Bass, 2017.

[5] Oracle Corporation, "The Java Tutorials and Java SE Documentation," https://docs.oracle.com/javase/tutorial/. Accessed October 2026.